

Visualisierung kinematischer Kennwerte

Philipp Mikos

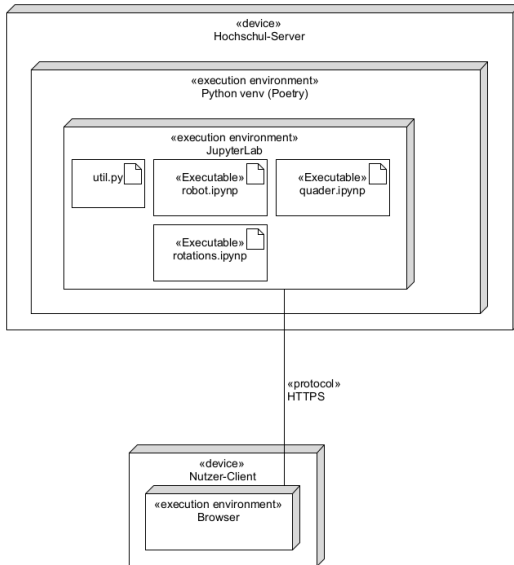
June 9, 2025

- 1 Einleitung
- 2 Anforderungen
- 3 Aufbau
- 4 Evaluation
- 5 Ausblick auf die Bachelorarbeit

- Entwicklung von Framework zur Visualisierung kinematischer Kennwerte für die Lehre
- Mathematische Konzepte für Studenten schwierig zu durchdringen
- Besonderer Fokus: Rotation von Körpern
- Werkzeug für Lehrpersonen zum erstellen von Übungsszenarien

- Transformationen von Körpern im Raum Visualisieren (insbesondere Rotation)
- Darstellungsmöglichkeiten (Euler, Matrix, Quaternionen)
- Komfortable Erstellung und Anwendung von Übungsaufgaben
- Einfache Bewegungsabläufe radgetriebener Roboter

- Python-Schnittstelle
- Nutzung innerhalb von JupyterLab
- Nach Möglichkeit Visualisierung innerhalb von JupyterLab
- Keine lokale Installation
- Kompatibel mit Sympy



Weitere Abhängigkeiten:

- Numpy
- SymPy
- SciPy
- pythreejs(3D Rendering)
- ipywidgets(GUI-Elemente)

util.py:

- Zentrale Datei util.py mit Kern- und Hilfsfunktionen
- Geometrische Transformationen und Umrechnung der Darstellungsformen von Rotationen
- Klasse Environment für konfigurierbare 3D-Szene mit Kamera, Licht und Gitter
- Funktion move() zur Simulation von Roboterbewegungen in der Ebene

Beispiel-Notebooks: `rotations.ipynb`

- Würfel der interaktiv per GUI-Elementen transformiert werden kann
- Möglichkeiten des Frameworks veranschaulichen
- Grundtransformationen nachvollziehbar machen

Beispiel-Notebooks: **quader.ipynb**

- Quader der anhand von Rotationsmatrizen rotiert werden soll
- Orientiert sich an einer Klausuraufgabe im Modul Robotik
- Visuelle Simulation der Rotationen

Beispiel-Notebooks: `robot.ipynb`

- Veranschaulichung einer Steuerungsaufgabe
- Orientiert sich an einer Übungsaufgabe im Modul Robotik
- Nutzer kann beobachten ob der Roboter dem gewünschten Pfad folgt

- Erfolgreiche Integration in JupyterLab: einfache Nutzung und direkte Visualisierung
- Unterstützung verschiedener Rotationsdarstellungen inkl. Umrechnungen
- Simulation radgetriebener Roboter über `move()`-Funktion realisiert
- Kompatibilität mit SymPy sichergestellt
- Zentrale Anforderungen erfüllt. Grundlage für spätere Erweiterungen geschaffen

- Deployment auf den JupyterLab-Servern der FH Dortmund
- Erweiterungen um Stationäre Roboter
- Vorwärts- und rückwärts-Transformation

**Vielen Dank für Ihre
Aufmerksamkeit!**